

Recursividade e memória

O que a recursão gasta, onde, e como não estourar

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
`andreyoshiaki@dcomp.ufs.br`

Até onde a pilha aguenta: o limite e o estouro



O que pesa em cada registro — e o que morre no `return`



Quando nada fica para a volta: recursão de cauda



Uma tabela no heap para todas as chamadas



`malloc` em cada chamada: quem libera, e quando



Exercícios

A pergunta que ficou

Da aula passada

Cada chamada ativa ocupa um **registro de ativação** na pilha, e nenhum é devolvido antes de a descida chegar ao caso base.

A pergunta que ficou: `soma(v, 10000000)`
funciona?

O que vamos descobrir

A pilha tem um tamanho fixo — e pequeno. Saber quanto cada chamada gasta, e o que mandar para o **heap**, é o que separa uma função recursiva que funciona de uma que derruba o programa.

A pergunta de hoje

Quanta memória uma função recursiva gasta, em que lugar, e o que fazer quando ela não cabe?

```
/* memoria.c --- quanto de memoria a recursao gasta, e onde */
#include <stdio.h>
#include <stdlib.h>
/* A: a soma da aula passada: a adicao fica para a volta */
long soma(const int *v, int n)
{
    if (n == 0)
        return 0;
    return soma(v, n - 1) + v[n - 1];
}

/* B: a mesma soma, levando o parcial na ida: nada fica para a volta */
long soma_acum(const int *v, int n, long parcial)
{
    if (n == 0)
        return parcial;
    return soma_acum(v, n - 1, parcial + v[n - 1]);
}
```

```
/* C: Fibonacci que anota cada resultado em uma tabela no heap */
long fib_memo(int n, long *memo)
{
    if (n < 2)
        return n;
    if (memo[n] == 0)
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo);
    return memo[n];
}
```

Lembrete da aula passada

A fib ingênua recalculava os mesmos valores do zero: fib(40) fez 331 160 281 chamadas. Aqui, cada fib(k) calculado fica guardado em memo[k].

```
int main(int argc, char *argv[])
{
    int n = (argc > 1) ? atoi(argv[1]) : 1000;
    int *v = malloc(n * sizeof *v);      /* o vetor mora no heap */
    long *memo = calloc(91, sizeof *memo); /* tabela zerada */
    if (v == NULL || memo == NULL) {
        free(v);
        free(memo);
        return 1;
    }
    for (int i = 0; i < n; i++)
        v[i] = 1;
    printf("C: fib(90) = %ld\n", fib_memo(90, memo));
    free(memo);

    printf("B: soma_acum(v, %d) = %ld\n", n, soma_acum(v, n, 0));
    printf("A: soma(v, %d) = %ld\n", n, soma(v, n));
    free(v);
    return 0;
}
```

Antes de compilar, escreva sua resposta

1. Com `./memoria 100000`: quais linhas aparecem, e com que valores?
2. E com `./memoria 1000000`? E com `10000000`?
3. Se alguma execução falhar, qual linha sai por último?
4. A linha **C** demora? A `fib` ingênua levaria quanto tempo para `fib(90)`?
5. O vetor de 10 milhões de `int` ocupa 40 MB. Ele é o problema?

```
$ gcc -Wall -Wextra memoria.c -o memoria
$ ./memoria 100000
C: fib(90) = 2880067194370816120
B: soma_acum(v, 100000) = 100000
A: soma(v, 100000) = 100000
$ ./memoria 1000000
C: fib(90) = 2880067194370816120
Segmentation fault: 11
$ echo $?
139
```

No Linux a mensagem é `Segmentation fault (core dumped)`. Com 10000000, o mesmo resultado.

Até onde a pilha aguenta

- a linha **C** saiu: o heap deu conta da tabela;
- a linha **B** *não* saiu: o programa morreu *dentro* de `soma_acum`;
- o vetor de 1 milhão de `int` (4 MB) foi alocado sem problema.

Pense antes de passar o slide

O que `soma_acum(v, 1000000, 0)` empilha que `soma_acum(v, 100000, 0)` não empilha?

Existe um n exato a partir do qual o programa cai? Do que ele depende?

Limite da pilha

Cada programa recebe, ao iniciar, uma pilha de tamanho **fixo**, definido pelo sistema operacional. Em Linux e macOS o padrão é de cerca de **8 MB**; o comando `ulimit -s` mostra o valor em KB.

Estouro de pilha (*stack overflow*)

Quando os registros ativos passam desse limite, o próximo registro cai em memória que não pertence à pilha. O sistema interrompe o programa (SIGSEGV, *segmentation fault*).

Intuição

profundidade máxima $\approx$ tamanho da pilha $\div$ tamanho de um registro

Fonte: Kerrisk (2010), §36.3 [8]; Bryant & O'Hallaron (2015), §3.7.1 [2]

A conta: quantos registros cabem em 8 MB



n	pilha necessária	cabe?
1 000	48 KB	sim
100 000	4,8 MB	sim
1 000 000	48 MB	não
10 000 000	480 MB	não

registros de 48 bytes, compilado sem otimização

Nesta máquina, `ulimit -s` dá 8 176 KB
= 8 372 224 bytes.

Por busca binária, o maior n que passa
fica em torno de **174 mil** (varia um pouco
a cada execução):

$8\,372\,224 \div 174\,000 \approx 48$ bytes por registro.

O limite não é um número mágico: é uma
divisão.

Fonte: Kerrisk (2010), §36.3 [8]; medido com `memoria.c`

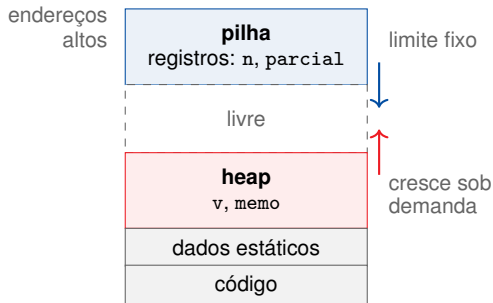
Pense antes de passar o slide

No `main`, `v` foi criado com `malloc`. O que aconteceria se ele fosse declarado assim?

```
int v[100000000];
```

E por que 40 MB no heap não são problema, se 48 MB na pilha são?

A resposta: pilha e heap são regiões diferentes



`int v[10000000]` como local vai para o registro do main: 40 MB numa pilha de 8 MB. O programa cai **antes** da primeira chamada.

O heap cresce sob demanda, até o limite da memória do processo. Se não houver espaço, `malloc` devolve `NULL` — um erro que dá para *testar*. O estouro de pilha não avisa.

Fonte: Kerrisk (2010), §6.3 [8]; Bryant & O'Hallaron (2015), §9.9 [2]

O que aprendemos

- a pilha tem tamanho fixo, da ordem de 8 MB;
- a profundidade máxima é o limite dividido pelo tamanho de um registro;
- passar do limite é um *segmentation fault*, sem aviso e sem chance de recuperação;
- dados grandes vão para o heap, onde a falta de espaço vira um NULL testável.

O que pesa em cada registro

```
/* gorda.c */
int desce(int n)
{
    char linha[4096];    /* 4 KB */
    linha[0] = (char) n;
    if (n == 0)
        return linha[0];
    return desce(n - 1) + linha[0];
}
```

Pense antes de passar o slide

soma_acum aguentou cerca de 174 mil níveis. Quantos níveis desce aguenta?

- uns 170 mil?
- uns 20 mil?
- uns 2 mil?

Medido: desce cai a partir de **2010** níveis.

$$8\,372\,224 \div 2\,010 \approx 4\,165 \text{ bytes}$$

= os 4 096 de linha + o resto do registro.

Um vetor local existe uma vez *por chamada ativa*. Em uma função recursiva, ele é pago a cada nível.

O que fazer

- alocar o espaço **uma vez**, no heap, fora da recursão;
- passar o ponteiro para baixo como parâmetro;
- cada registro paga só 8 bytes do ponteiro.

Fonte: Bryant & O'Hallaron (2015), §3.7.4 [2]; medido com `gorda.c`

E se a função devolver o endereço de uma local?

```
char *rotulo(int n)
{
    char texto[16];
    snprintf(texto, sizeof texto,
              "nivel %d", n);
    return texto;
}

int main(void)
{
    char *a = rotulo(1);
    char *b = rotulo(2);
    printf("%s | %s\n", a, b);
    return 0;
}
```

Pense antes de passar o slide

O que o printf imprime?

Onde está texto depois que rotulo retorna?

```
$ gcc -Wall -Wextra rotulo.c -o rotulo
rotulo.c:6:12: warning: address of stack memory associated with local
      variable 'texto' returned [-Wreturn-stack-address]
$ ./rotulo
??? | ???      <- bytes ilegíveis, outros a cada execução
```

A vida de uma local termina quando a chamada termina. O ponteiro continua apontando para onde ela estava — uma área que a próxima chamada reaproveita. O resultado é imprevisível; o compilador avisa (no gcc: `-Wreturn-local-addr`).

Para devolver texto: receba o espaço do chamador, ou devolva memória do `malloc` — e documente quem dá `free`.

Fonte: ISO/IEC 9899:2018, §6.2.4 [6]; Bryant & O'Hallaron (2015), §9.11 [2]

O que aprendemos

- parâmetros e locais — inclusive vetores — moram no registro;
- numa função recursiva, cada local é paga uma vez por nível;
- espaço grande: aloque uma vez no heap e passe o ponteiro;
- o registro deixa de existir no `return`: nunca devolva o endereço de uma local.

Recursão de cauda

Por que, com -O2, a linha B passou?



```
$ gcc -O2 -Wall -Wextra memoria.c -o memoria
$ ./memoria 1000000
C: fib(90) = 2880067194370816120
B: soma_acum(v, 1000000) = 1000000
Segmentation fault: 11
```

Pense antes de passar o slide

Mesmo programa, mesmo n . Só mudou a opção de otimização.

Por que `soma_acum` agora aguenta um milhão de níveis e `soma` não? O que uma tem que a outra não tem?

Chamada de cauda

Uma chamada é **de cauda** quando é a última coisa que a função faz: o valor que ela devolve é devolvido direto, sem nenhuma conta depois. Uma função recursiva cujas chamadas são todas de cauda é uma **recursão de cauda**.

Intuição

Se nada fica pendente, o registro de quem chamou não serve para mais nada: o compilador pode *reaproveitá-lo* e trocar a chamada por um salto — a recursão vira um laço.

```
return soma(v, n - 1) + v[n - 1];    a soma fica para depois  
return soma_acum(v, n - 1, parcial + v[n - 1]);  nada fica
```

Fonte: Abelson et al. (1996), §1.2.1 [1]

O que fica pendente em cada registro

soma(v, 3)

n = 0	devolve 0
n = 1	espera ? + v[0]
n = 2	espera ? + v[1]
n = 3	espera ? + v[2]
main	

cada registro guarda uma conta por terminar:
todos precisam continuar vivos

soma_acum(v, 3, 0)

n	parcial
main	
n	parcial
3	0
2	1
1	2
0	3 devolve 3

o resultado parcial desce junto:
um registro só, atualizado no lugar

Fonte: Abelson et al. (1996), §1.2.1 [1]

O que você escreveu

```
long soma_acum(const int *v, int n,
               long parcial)
{
    if (n == 0)
        return parcial;
    return soma_acum(v, n - 1,
                    parcial + v[n - 1])
    ;
}
```

O que o -O2 gerou (em C)

```
long soma_acum(const int *v, int n,
               long parcial)
{
    while (n != 0) {
        parcial = parcial + v[n - 1];
        n = n - 1;
    }
    return parcial;
}
```

Os parâmetros viram as variáveis do laço; a chamada vira “volte ao início”. Um registro só, qualquer que seja n .

Fonte: Abelson et al. (1996), §1.2.1 [1]; Bryant & O'Hallaron (2015), §3.7 [2]

A otimização não é garantida

- o padrão C não exige que chamadas de cauda virem laço;
- sem `-O2` — como na depuração — `soma_acum` caiu em cerca de 174 mil níveis;
- outro compilador ou outra versão pode fazer diferente, para mais ou para menos;
- uma mudança pequena (um `printf` depois da chamada) tira a chamada da cauda.

Regra prática

Se a profundidade pode chegar a dezenas de milhares, escreva o laço você mesmo. A versão de cauda é o meio do caminho: já tem a cara do laço.

Fonte: ISO/IEC 9899:2018, §6.5.2.2 [6]; medido com `memoria.c` (clang, macOS)

O que aprendemos

- numa chamada de cauda nada fica pendente no registro de quem chamou;
- o resultado parcial desce como parâmetro (um acumulador);
- com otimização, o compilador *pode* transformar a recursão em laço;
- “pode” não é “vai”: para profundidades grandes, use o laço.

**Uma tabela para todas as
chamadas**

Por que `fib(90)` saiu na hora?



A `fib` ingênua faz $2F(n+1) - 1$ chamadas.
Para $n = 90$:

$$\approx 9,3 \times 10^{18} \text{ chamadas}$$

A um bilhão de chamadas por segundo,
quase **300 anos**.

Pense antes de passar o slide

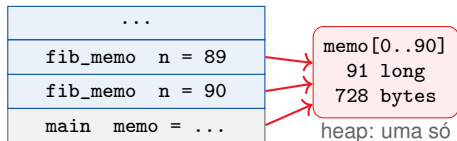
Quantas chamadas `fib_memo(90, memo)` faz?

Quantas tabelas `memo` existem durante a execução — uma por chamada, ou uma só?

Fonte: Deitel & Deitel (2011), §5.15 [4]

Memoização

Guardar o resultado de cada chamada já calculada e, na próxima vez que ela for pedida, **consultar** em vez de recalcular.



Cada registro tem só uma *cópia do ponteiro*; a tabela é uma só.
Resultado: **179** chamadas ($2n - 1$)
em vez de $9,3 \times 10^{18}$.

Fonte: Michie (1968) [10]; Cormen et al. (2009), §15.3 [3]

Fora da recursão

A tabela precisa existir **antes** da primeira chamada e **depois** da última. Por isso o `main` aloca, passa o ponteiro e libera no fim.

Por que `calloc`

O zero significa “ainda não calculado”. Com `malloc`, a tabela viria com lixo — e o lixo seria lido como resultado pronto.

é de...	vai para	exemplo
uma chamada só	a pilha (local)	<code>n</code> , <code>partial</code>
todas as chamadas	o heap, alocado fora	<code>memo</code> , <code>v</code>

Fonte: Kernighan & Ritchie (1989), §7.8.5 [7]; Cormen et al. (2009), §15.3 [3]

O que aprendemos

- memoização troca memória por tempo: 728 bytes no lugar de séculos;
- o que é compartilhado por todas as chamadas vai para o heap, alocado fora da recursão;
- cada registro carrega só o ponteiro, não a tabela;
- `calloc` dá uma tabela zerada — e o zero precisa ter significado.

`malloc` **em cada chamada**

```
/* invertida.c: copia de s de tras para frente */
char *invertida(const char *s)
{
    if (*s == '\0') {                               /* caso base */
        char *r = aloca(1);
        r[0] = '\0';
        return r;
    }
    char *resto = invertida(s + 1);
    size_t k = strlen(resto);
    char *r = aloca(k + 2);
    strcpy(r, resto);    /* o resto invertido... */
    r[k] = *s;           /* ...e a minha letra */
    r[k + 1] = '\0';
    libera(resto);      /* na volta */
    return r;
}
```

aloca e libera chamam
malloc e free, e contam
quantos bytes estão no heap
agora e no máximo (o *pico*).

Cada nível recebe um bloco da
chamada de baixo, monta um
bloco maior e devolve.

Antes de executar, escreva sua resposta

1. Para uma string de $n = 10\,000$ letras, quantos bytes ficam no heap no **pico**? Uns 20 mil? Uns 10 milhões?
2. E se apagarmos a linha `libera(resto);`? Quanto fica no heap no fim?
3. Sem o `libera`, o resultado impresso muda?

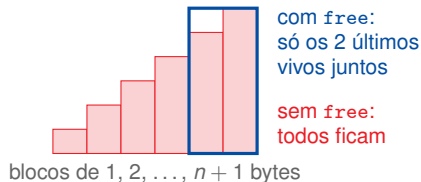
<i>n</i>	com libera(resto)	sem libera(resto)	
	pico	pico	perdido no fim
10	21	66	55
1 000	2 001	501 501	500 500
10 000	20 001	50 015 001	50 005 000

Sem o `libera`, o resultado impresso é **o mesmo** — nada avisa. Para inverter 10 KB de texto, o programa deixa 50 MB perdidos no heap.

“perdido” = pico menos o bloco do resultado, que o `main` ainda usa e libera.

Fonte: medido com `invertida.c`

Por que $2n$ com o free e $n^2/2$ sem ele



Com o free, enquanto um nível monta o seu bloco ($k + 2$ bytes), só o do nível de baixo ($k + 1$) ainda existe: pico $\approx 2n$.

Sem o free, nenhum bloco sai:

$$1 + 2 + \dots + (n + 1) = \frac{(n+1)(n+2)}{2}$$

É um **vazamento de memória** que cresce com o quadrado de n .

Fonte: King (2008), §17.4 [9]; Bryant & O'Hallaron (2015), §9.11 [2]

O contrato

invertida devolve um bloco do heap.
Quem chama *recebe a posse*: precisa usá-lo e depois dar `free` — seja o `main`, seja o nível de cima da própria recursão.

Ida e volta, de novo

O `libera(resto)` vem **depois** da chamada: roda na volta, quando o bloco já foi copiado. Antes do `strcpy`, seria um uso de memória já liberada.

Escreva o contrato no comentário da função:

```
/* devolve bloco do heap; quem chama deve dar free */
```

Fonte: Kernighan & Ritchie (1989), §7.8.5 [7]; King (2008), §17.4 [9]

O que aprendemos

- cada nível que aloca precisa liberar o que não vai devolver;
- o `free` do bloco recebido vem depois de usá-lo — na volta;
- um vazamento dentro da recursão se multiplica pelo número de chamadas;
- o resultado certo não prova que a memória está certa: meça.

Mãos à obra

Tarefa 1

Descubra o limite da *sua* máquina: o maior n que `./memoria` aceita sem e com `-O2`. Rode `ulimit -s` e calcule o tamanho de cada registro.

Tarefa 2

Rode `ulimit -s 65536` e repita `./memoria 1000000`. Depois ponha um `int` rascunho[32] (usado) em `soma_acum`. Preveja o novo limite antes de medir.

Tarefa 3

Troque `calloc` por `malloc` na tabela. O que muda? E se a tabela tiver 90 posições em vez de 91?

Exercício 1 — somar pelas metades

Escreva `long soma_metades(const int *v, int ini, int fim)` que soma `v[ini..fim-1]` somando a metade esquerda e a direita. Ela funciona com 10 milhões de elementos sem -O2. Por quê?

Exercício 2 — o vetor auxiliar vai para o heap

Uma ordenação recursiva declara `int aux[fim - ini]` a cada chamada e cai com 10 milhões de elementos. Aloque `aux` **uma vez**, no heap, e passe-o para baixo.

Fonte: Feofiloff (2009), cap. 2 [5]

Exercício 3 — máximo com acumulador

`maximo` em recursão de cauda, levando o maior até agora como parâmetro. Teste com 1 milhão de elementos, sem e com -02.

Exercício 4 — combinações com tabela

$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$. Calcule $C(60, 30)$ com uma tabela $(n + 1) \times (k + 1)$ em bloco único no heap (`memo[i * nc + j]`).

Exercício 5 — repetir sem vaziar

`char *repete(const char *s, int k)` devolve `s` repetida `k` vezes, num bloco do heap. No fim, o heap volta a zero.

Fonte: Cormen et al. (2009), §15.3 [3]; King (2008), §17.4 [9]

Pintar uma região: troque a pilha do sistema por uma sua

Uma grade de $1 \times c$ caracteres, alocada no heap, tem células livres (.) e paredes (x).

- `pinta_rec`: a partir de (i, j) , marca a célula com # e chama a si mesma para os quatro vizinhos; devolve quantas pintou;
- `pinta_pilha`: o mesmo, sem recursão, com uma pilha de posições no heap que cresce com `realloc`.
- as duas concordam em grades pequenas;
- numa grade 3000×3000 , a recursiva cai — mesmo com -O2. Explique por quê;
- a versão com pilha própria pinta as 4 500 000 células e libera tudo.

Fonte: Bryant & O'Hallaron (2015), §9.9 [2]

O que mudou hoje

A pergunta da aula passada tem resposta: dez milhões de registros não cabem. Mas a soma de dez milhões cabe — em um laço, ou em uma recursão que desce só $\log_2 n$ níveis.

O preço

A pilha é pequena, e estourá-la não avisa. O heap é grande, mas cada bloco precisa de um dono que dê `free`.

Para levar

- profundidade máxima $\approx$ pilha $\div$ registro;
- vetores locais pesam em cada nível: aloque fora e passe o ponteiro;
- recursão de cauda *pode* virar laço — não conte com isso;
- o que é de todas as chamadas vai para o heap; quem recebe um bloco o libera.

-  Harold Abelson, Gerald Jay Sussman, and Julie Sussman.
Structure and Interpretation of Computer Programs.
MIT Press, Cambridge, MA, 2 edition, 1996.
Seção 1.2.1: recursão linear e iteração.
-  Randal E. Bryant and David R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Pearson, Boston, 3 edition, 2015.
Seções 3.7 (a pilha de execução), 9.9 (alocação dinâmica) e 9.11 (erros comuns de memória).

 Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Introduction to Algorithms.

MIT Press, Cambridge, MA, 3 edition, 2009.

Seção 15.3: memoização.

 Paul Deitel and Harvey Deitel.

C: como programar.

Pearson Prentice Hall, São Paulo, 6 edition, 2011.

Capítulo 5: pilha de chamadas, Fibonacci e recursão versus iteração.

-  **Paulo Feofiloff.**
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.
Capítulo 2: algoritmos recursivos.

-  **ISO/IEC.**
ISO/IEC 9899:2018 — Information technology — Programming languages — C.
Technical report, International Organization for Standardization, 2018.
Seções 6.2.4 (duração de armazenamento) e 7.22.3 (gerenciamento de memória).

-  Brian W. Kernighan and Dennis M. Ritchie.
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.
Seções 4.10 (recursividade) e 7.8.5 (malloc, calloc e free).
-  Michael Kerrisk.
The Linux Programming Interface.
No Starch Press, San Francisco, 2010.
Seções 6.3 (layout de memória de um processo) e 36.3 (RLIMIT_STACK).



K. N. King.

C Programming: A Modern Approach.

W. W. Norton, New York, 2 edition, 2008.

Seções 9.6 (recursion) e 17.4 (deallocating storage).



Donald Michie.

“memo” functions and machine learning.

Nature, 218:19–22, 1968.